Chapter 7. Convolutional Neural Networks and Architectures

Convolution, stride, padding, pooling, equivariance; AlexNet, LRN, ZFNet, VGG, GoogLeNet/Inception, ResNet, saliency maps

Contents

1 Why a New Layer **2**

2 Convolution as a Local Dot Product **2**

2.1 A fixed filter: moving average 2

2.2 Single and multiple channels 2

2.3 Parameter counts 3

3 Stride, Padding and Output Size **3**

4 Learning Filters **4**

5 Shift Equivariance **4**

6 1×1 Convolution, Pooling and Global Average Pooling **4**

7 Algebraic Facts about CNN Layers **4**

8 Vocabulary and Costs **5**

9 AlexNet **5**

10 ZFNet and VGG **6**

11 GoogLeNet and the Inception Module **6**

11.1 Naive module 6

11.2 Bottleneck module 6

11.3 Split–transform–merge 7

12 Why Depth Can Fail, and Residual Networks **7**

13 Saliency Maps **8**

1 Why a New Layer

A fully connected layer on flattened $X \in \mathbb{R}^{H\times W\times C}$ has $M(HWC+1)$ parameters (3,073,000 for $32\times 32\times 3$, $M=1000$) and ignores neighborhoods. Convolution assumes:

- **Locality**: edges and textures live in small patches (local connectivity).
- **Stationarity**: a pattern may occur anywhere (weight sharing).

Definition 7.1: equivariance and invariance

Let $T_{\boldsymbol{\tau}}$ shift an image, $(T_{\boldsymbol{\tau}}X)_{i,j,c} = X_{i-\tau_1,j-\tau_2,c}$. F is shift *equivariant* if $F(T_{\boldsymbol{\tau}}X) = T_{\boldsymbol{\tau}}F(X)$ and G is shift *invariant* if $G(T_{\boldsymbol{\tau}}X) = G(X)$. Generally, for a group acting by $\rho_{\text{in}}(g)$, $\rho_{\text{out}}(g)$, f is equivariant if $f(\rho_{\text{in}}(g)x) = \rho_{\text{out}}(g)f(x)$; invariance is the case $\rho_{\text{out}} = \text{id}$.

2 Convolution as a Local Dot Product

2.1 A fixed filter: moving average

With the 3×3 averaging kernel (1/9 everywhere), $Y_{i,j} = \frac{1}{9}\sum_{u,v=0}^{2} X_{i+u,j+v}$: on a 0/90 image a window with r bright pixels gives $10r$; isolated pixels are suppressed, edges blurred. A CNN *learns* the kernel (any signs, any sum).

2.2 Single and multiple channels

Definition 7.2: convolutional layer

With D filters $W \in \mathbb{R}^{D\times K\times K\times C}$, biases $\mathbf{b}\in\mathbb{R}^D$, stride S, padding P,

$$Z_{i,j,d} = b_d + \sum_{u=0}^{K-1}\sum_{v=0}^{K-1}\sum_{c=1}^{C} W_{d,u,v,c}\,\tilde{X}_{Si+u,\,Sj+v,\,c},$$

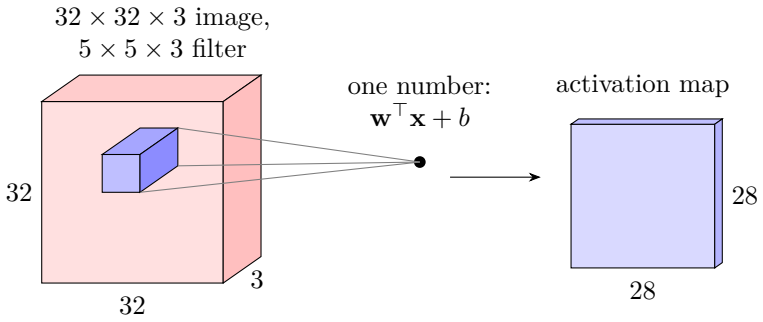
$\tilde{X}$ padded. Each output is a patch–filter dot product over *all* input channels plus bias; $Z_{\cdot,\cdot,d}$ is the feature map of filter d. (Cross-correlation, no flip; irrelevant for learned kernels.)

Example 7.3: one channel, 2×2 filter

$X = \begin{pmatrix} 1 & 2 & 0\\ 0 & 1 & 3\\ 2 & 1 & 0\end{pmatrix}$, $W = \begin{pmatrix} 1 & 0\\ -1 & 2\end{pmatrix}$, $b=1$, $S=1$, $P=0$: $Z = \begin{pmatrix} 4 & 8\\ 1 & 1\end{pmatrix}$, same weights at all four positions.

Example 7.4: a 5×5 example

With a 5×5 image and $W = \begin{pmatrix} 1 & 2 & 3\\ -4 & 7 & 4\\ 2 & -5 & 1\end{pmatrix}$ $(b=0)$: unpadded $Z = \begin{pmatrix} 51 & 66 & 20\\ 31 & 49 & 101\\ 15 & 53 & -2\end{pmatrix}$; with $P=1$ the 5×5 output has this as its central block.



A full-depth filter slid over all positions gives one 28×28 *map.*

2.3 Parameter counts

Six $5 \times 5 \times 3$ filters: $32 \times 32 \times 3 \to 28 \times 28 \times 6$ with $6(75+1) = 456$ parameters. Same shapes:

Layer	Parameters
Fully connected	$4704(3072+1) = 14{,}455{,}392$
Locally connected, unshared	$28^2 \cdot 6(75+1) = 357{,}504$
Convolution (shared)	$6(75+1) = 456$

As a matrix, convolution is sparse and banded with repeated entries (filter (a, b) on $(x_1, \ldots, x_4)$: two free numbers).

3 Stride, Padding and Output Size

Motivation. One formula gives every layer's output shape.

Theorem 7.5: output size

For input length N, kernel K, stride S, padding P (symmetric), the number of output positions is

$$N' = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1, \qquad C' = D.$$

Proof. A window starting at tS needs $tS + K - 1 \le N + 2P - 1$, i.e. $t = 0, \ldots, \lfloor (N + 2P - K)/S \rfloor$. ■

$N = 7$, $K = 3$: $S = 1, 2, 3 \Rightarrow 5, 3, 2$ (for $S = 3$ a border is unused). $S = 1$, odd K: $P = (K-1)/2$ preserves size. Zero padding is a boundary assumption.

Example 7.6: further examples

Input $52 \times 52 \times 128$, $D = 256$.
(1) $K = 3, S = 1, P = 1$: output $52 \times 52 \times 256$, parameters $256(9 \cdot 128 + 1) = 295{,}168$; each filter is $3 \times 3 \times 128$.
(2) $K = 4, S = 2, P = 1$: $\lfloor (52 + 2 - 4)/2 \rfloor + 1 = 26$, output $26 \times 26 \times 256$, parameters $256(16 \cdot 128 + 1) = 524{,}544$.

Stacking $32 \times 32 \times 3 \to 28 \times 28 \times 6 \to 24 \times 24 \times 10$ (ten $5 \times 5 \times 6$ filters, 1510 parameters); two stride-1 5×5 layers see 9×9. With strides, $r_\ell = r_{\ell-1} + (K_\ell - 1) j_{\ell-1}$ and $j_\ell = S_\ell j_{\ell-1}$, starting from $r_0 = j_0 = 1$.

4 Learning Filters

Unpadded, stride 1, $G_{i,j,d} = \partial L/\partial Z_{i,j,d}$:

$$\frac{\partial L}{\partial W_{d,u,v,c}} = \sum_{i,j} G_{i,j,d} X_{i+u,j+v,c}, \qquad \frac{\partial L}{\partial b_d} = \sum_{i,j} G_{i,j,d} :$$

every position updates the shared weight.

5 Shift Equivariance

Motivation. Convolution shifts features with the image.

Theorem 7.7: convolution commutes with shifts

On an infinite grid (or with circular indices), stride-1 convolution satisfies $C_W(T_{\boldsymbol{\tau}} X) = T_{\boldsymbol{\tau}} C_W(X)$, and pointwise nonlinearities also commute with shifts. Global average pooling of a circularly shifted map is unchanged, so GAP $\circ$ (convolutions, ReLUs) is shift invariant.

Proof. $[C_W(T_{\boldsymbol{\tau}} X)]_{i,j,d} = b_d + \sum_{u,v,c} W_{d,u,v,c} X_{i+u-\tau_1, j+v-\tau_2, c} = [C_W X]_{i-\tau_1, j-\tau_2, d}$. A circular shift permutes positions; averages are permutation invariant. ■

Remark 7.8: when exactness fails

Zero padding breaks equality near the border. Stride S (or pooling) commutes only with shifts by multiples of S: for width-2, stride-2 max pooling M, $M(T_2 x) = T_1 M(x)$ but $M(T_1 x)$ need not be a shift of $M(x)$ (e.g. $x = (0, 1, 2, 0)$, $M(x) = (1, 2)$, $M(T_1 x) = (0, 2)$).

6 1×1 Convolution, Pooling and Global Average Pooling

1×1 convolution. $\mathbf{y}_{i,j} = A\mathbf{x}_{i,j} + \mathbf{b}$, $A \in \mathbb{R}^{D \times C}$ shared: mixes channels, not neighbors. $56 \times 56 \times 64 \to 56 \times 56 \times 32$ costs $32(64 + 1) = 2080$ parameters.

Pooling. Channelwise, no parameters: $Y^{\max}_{i,j,c} = \max_{u,v} X_{iS+u, jS+v, c}$ or the average. For $X = \begin{pmatrix} 5 & 10 & 6 & 7 \\ 4 & 0 & 4 & 3 \\ 2 & 4 & 3 & 4 \\ 0 & 8 & 1 & 6 \end{pmatrix}$ and 2×2, stride 2: average $\begin{pmatrix} 4.75 & 5 \\ 3.5 & 3.5 \end{pmatrix}$, max $\begin{pmatrix} 10 & 7 \\ 8 & 6 \end{pmatrix}$. Pooling lowers resolution, with limited robustness to small shifts. Backward: max routes to the maximizer, average spreads g/K^2.

Global average pooling. $g_c = \frac{1}{HW} \sum_{i,j} X_{i,j,c}$, $H \times W \times C \to C$: the classifier needs $Q(C + 1)$, not $Q(HWC + 1)$, parameters; spatial arrangement is discarded.

7 Algebraic Facts about CNN Layers

Motivation. Identities saying when CNN layers can be reordered or merged.

Theorem 7.9: positive homogeneity

A network built only from bias-free convolutions and linear layers, ReLU, max pooling, reshaping and dropout satisfies $f(\lambda X) = \lambda f(X)$ for all $\lambda \geq 0$.

Proof. Each piece is positively homogeneous: linear maps; $\max(0, \lambda t) = \lambda \max(0, t)$, $\max_i \lambda x_i = \lambda \max_i x_i$ ($\lambda \geq 0$ preserves order and signs); dropout is a fixed mask. Compositions: $g_1(g_2(\lambda X)) = g_1(\lambda g_2(X)) = \lambda g_1(g_2(X))$. Biases break it (ReLU$(x+1)$ at $x = 0$). ■

Theorem 7.10: conv–MP–ReLU orders coincide

conv→MaxPool→ReLU, conv→ReLU→MaxPool and conv→ReLU→MaxPool→ReLU are the same function.

Proof. Nondecreasing ϕ gives $\max_i \phi(x_i) = \phi(\max_i x_i)$ (equality at the maximizer). ReLU is nondecreasing and elementwise, so ReLU $\circ$MP = MP $\circ$ ReLU, and ReLU $\circ$ ReLU = ReLU. (Average pooling: only avg ReLU $\geq$ ReLU avg, by convexity.) ■

Theorem 7.11: convolutions that implement given maps

With PyTorch's convention $Y_{o,i,j} = b_o + \sum_c \sum_{k,l=1}^{3} w_{o,c,k,l} \tilde{X}_{c,i+k-2,j+l-2}$ (3×3, padding 1), reading the value at offset $(\Delta i, \Delta j)$ requires a 1 at $(k, l) = (2 + \Delta i, 2 + \Delta j)$.

- Identity $C \to C$: $w_{c,c',k,l} = \delta_{cc'} \delta_{k2} \delta_{l2}$, $\mathbf{b} = \mathbf{0}$.
- Colorwise finite differences $Y_{2c-1} = X_{c,i+1,j} - X_{c,i,j}$, $Y_{2c} = X_{c,i,j+1} - X_{c,i,j}$: $w_{2c-1,c} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 1 & 0 \end{pmatrix}$, $w_{2c,c} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & -1 & 1 \\ 0 & 0 & 0 \end{pmatrix}$, all other slices zero; zero padding supplies the out-of-range zeros.

8 Vocabulary and Costs

$X \mapsto h(X) \mapsto \mathbf{s}(X) \in \mathbb{R}^Q \mapsto \mathbf{p}(X)$. A convolution with D filters $K \times K \times C$, output $H' \times W' \times D$:

$$N_{\text{param}} = D(K^2 C + 1), \qquad N_{\text{act}} = H'W'D, \qquad N_{\text{MAC}} \approx H'W'DK^2C.$$

FC $\mathbb{R}^n \to \mathbb{R}^m$: $m(n+1)$. Top-1 error: top score wrong; top-5: true label not among the top five.

9 AlexNet

Five convolutional and three FC layers (eight weight layers):

Stage	Settings	Output
Input	RGB	$227 \times 227 \times 3$
Conv1	96, 11×11, $S = 4$, $P = 0$	$55 \times 55 \times 96$ $\quad ((227 - 11)/4 + 1)$
Pool1, Norm1	3×3, $S = 2$; LRN	$27 \times 27 \times 96$
Conv2	256, 5×5, $P = 2$	$27 \times 27 \times 256$
Pool2, Norm2	3×3, $S = 2$; LRN	$13 \times 13 \times 256$
Conv3, Conv4	384, 3×3, $P = 1$	$13 \times 13 \times 384$
Conv5	256, 3×3, $P = 1$	$13 \times 13 \times 256$
Pool3	3×3, $S = 2$	$6 \times 6 \times 256 = 9216$
FC6, FC7, FC8	$9216 \to 4096 \to 4096 \to 1000$	1000 scores

The FC layers dominate: $4096 \cdot 9217 = 37{,}752{,}832$, $4096 \cdot 4097 = 16{,}781{,}312$, $1000 \cdot 4097 = 4{,}097{,}000$, together 58,631,144, versus $96(11^2 \cdot 3 + 1) = 34{,}944$ for Conv1. Design:

- **ReLU**: no positive-side saturation.
- **Large first stride**: $227 \to 55$ saves memory and compute.
- **Local response normalization**: $a_{i,j,c} \mapsto a_{i,j,c} / \big(k + \alpha \sum_{r \in N(c)} a_{i,j,r}^2\big)^{\beta}$ across neighboring channels.
- **Dropout** 0.5 in the FC layers.

10 ZFNet and VGG

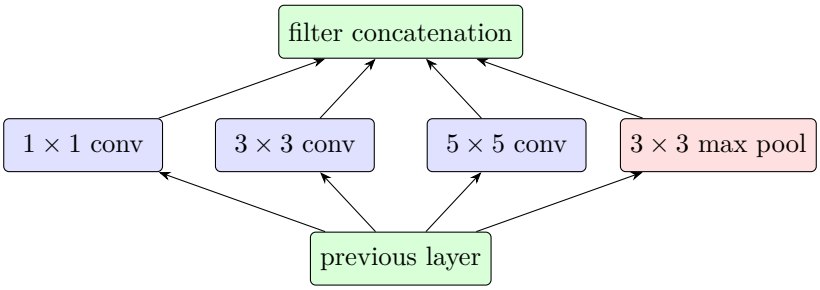
ZFNet retunes AlexNet's kernels and strides. VGG stacks 3×3 convolutions: two give a 5×5 field with $18C^2$ weights (vs. $25C^2$), three 7×7 with $27C^2$ (vs. $49C^2$), plus extra ReLUs. Receptive fields: $r_\ell = r_{\ell-1} + (K_\ell - 1) j_{\ell-1}$, $j_\ell = S_\ell j_{\ell-1}$.

11 GoogLeNet and the Inception Module

Motivation. Several filter sizes in parallel, concatenated, with 1×1 reductions to control cost.

11.1 Naive module

Input $28 \times 28 \times 256$, size-preserving branches: 1×1 (128), 3×3 (192), 5×5 (96), 3×3 max pool (stride 1). Channel concatenation gives $128 + 192 + 96 + 256 = 672$ channels. Weights: $256 \cdot 128 + 9 \cdot 256 \cdot 192 + 25 \cdot 256 \cdot 96 = 1{,}089{,}536$.



Naive Inception: parallel branches concatenated along depth.

11.2 Bottleneck module

1×1 reductions to 64 channels before the 3×3, 5×5 convolutions and after pooling: outputs $128 + 192 + 96 + 64 = 480$ channels and

$$32{,}768 + (16{,}384 + 110{,}592) + (16{,}384 + 153{,}600) + 16{,}384 = 346{,}112 \text{ weights}$$

(about 330k), a 68% reduction.

Theorem 7.12: when a bottleneck saves weights

A direct $K \times K$ map $C \to D$ costs K^2CD; reducing to R channels first costs $CR + K^2RD$. This is cheaper iff $R < \dfrac{K^2CD}{C + K^2D}$ (for $K = 5, C = 256, D = 96$ the threshold is ≈ 231).

GoogLeNet stacks such modules (22 weight layers) and ends with global average pooling instead of a large FC stack.

11.3 Split–transform–merge

A $256 \to 128 \to 128 \to 256$ block (1×1, 3×3 padding 1, 1×1, biases) has $32{,}896+147{,}584+33{,}024 = 213{,}504$ parameters; thirty-two summed $256 \to 4 \to 4 \to 256$ paths have $32(1028 + 148 + 1280) = 78{,}592$ (the 3×3 cost drops by 32). Implementation: three `nn.ModuleLists` of 32 convolutions and a sum, or `groups=32`.

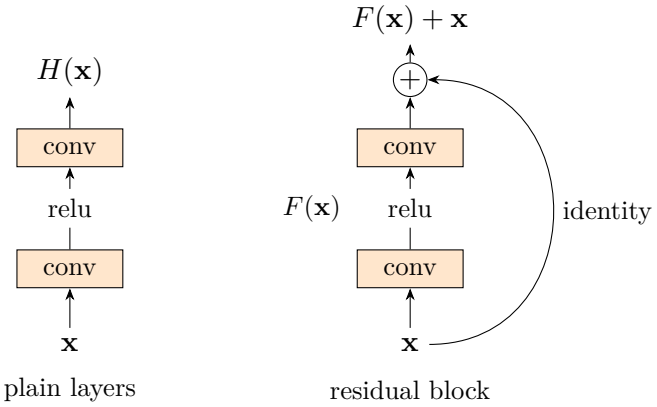
12 Why Depth Can Fail, and Residual Networks

Deeper plain networks can have *higher training error*: an optimization (degradation) problem, not overfitting, since with identity layers the optimal loss is no worse. For $\mathbf{x}_{\ell+1} = f_\ell(\mathbf{x}_\ell)$, $\nabla_{\mathbf{x}_0} L = J_0^\top \cdots J_{L-1}^\top \nabla_{\mathbf{x}_L} L$; Jacobian norms $\le \rho < 1$ shrink it like ρ^L ($0.9^{50} \approx 0.005$).

Theorem 7.13: residual block

$\mathbf{y} = \mathbf{x} + F(\mathbf{x}; \boldsymbol{\theta})$ gives $\dfrac{\partial \mathbf{y}}{\partial \mathbf{x}} = \mathbf{I} + J_F(\mathbf{x})$ and $\nabla_{\mathbf{x}} L = \mathbf{g} + J_F^\top \mathbf{g}$ with $\mathbf{g} = \nabla_{\mathbf{y}} L$.

The identity path carries the gradient when J_F is small, and $F = 0$ is the identity: the block learns $F = H - \mathrm{id}$. Not a guarantee ($J_F = -\mathbf{I}$ cancels; a ReLU after the sum alters the identity). Shape changes use $\mathbf{y} = P(\mathbf{x}) + F(\mathbf{x})$ (1×1 or strided convolution); addition keeps the channel count. Bottleneck $256 \to 64 \to 64 \to 256$: $2 \cdot 256 \cdot 64 + 9 \cdot 64^2 = 69{,}632$ weights vs. $18 \cdot 256^2 = 1{,}179{,}648$ for two full 3×3 layers. On CIFAR-10, residual nets remove the degradation; residual-MLP backpropagation: Chapter 5.



A residual block learns $F(\mathbf{x}) = H(\mathbf{x}) - \mathbf{x}$; gradients pass through the shortcut.

13 Saliency Maps

Differentiate the class score in the input (not the weights):

$$G_{i,j,k} = \frac{\partial s_c(X)}{\partial X_{i,j,k}}, \qquad M_{i,j} = \max_k |G_{i,j,k}|.$$

By Taylor, $s_c(X + \Delta X) \approx s_c(X) + \langle \nabla_X s_c(X), \Delta X \rangle$: large $|G|$ means high sensitivity. For $s(\mathbf{x}) = 2x_1 - 3x_2 + 0.1x_3$, x_2 ranks first. Saliency is local sensitivity, not causal explanation.